

Question 1: Solution.

Part- A

Flaw 1: Hardcoded Payment Logic is incompatible with AI-driven dynamic strategies

- The `OrderService` uses static `if-else` conditions to determine payment execution, assuming a **fixed and known set of payment types**.
- In the AI-assisted system, payment methods are **not predetermined**; they are dynamically suggested by the LLM based on context such as user behavior, vendor configuration, or transaction risk.
- This creates a mismatch: the backend expects deterministic inputs, while the AI produces **open-ended and evolving outputs** (e.g., BNPL, split payments, vendor-specific gateways).
- As a result, every new AI-suggested payment strategy forces modification in `OrderService`, tightly coupling business logic with AI variability.

Constraint violated:

(1) Different vendors may require different payment gateways

- Since vendors may have unique gateway mappings, hardcoding prevents adapting to vendor-specific AI recommendations without repeated code changes.

AI-specific issue:

The design fails to handle the **non-deterministic and evolving nature of AI-generated decisions**, making it unsuitable for AI-assisted systems.

Flaw 2: Absence of a validation layer between AI decisions and core business logic

- The system directly feeds AI outputs into `OrderService` without any validation or rule enforcement.
- However, LLM outputs are inherently **probabilistic and may be incorrect, inconsistent, or contextually invalid**.
- For example, AI may recommend a payment method not supported by the vendor or suggest an unsafe transaction flow.
- Without a validation layer, incorrect AI outputs directly impact critical operations like payments and discounts, leading to business risks.

Constraint violated:

(5) Avoid performance overhead and poor design due to abstraction misuse

- Without separation, validation logic would either:
 - Be duplicated across services (causing complexity), OR
 - Be skipped entirely (causing unsafe execution)

AI-specific issue:

The design assumes AI outputs are **deterministic and reliable**, whereas in reality, they require **guardrails and post-processing before execution**.

Flaw 3: Notification and logging logic are tightly coupled and not adaptable to AI-driven personalization

- The current system does not provide abstractions for notification channels or logging mechanisms, embedding them implicitly within the order flow.
- In the AI-enhanced system, notification channels are selected dynamically based on user preferences (Email, SMS, WhatsApp), and logging requirements may vary by vendor.
- This results in frequent modifications to core logic whenever AI introduces new communication strategies or vendor-specific requirements.

Constraints violated:

(2) Notifications must support Email, SMS, WhatsApp

(3) Some vendors require audit logging for every transaction

- The system cannot flexibly adapt to AI-driven personalization or vendor-specific compliance needs.

AI-specific issue:

The design cannot accommodate **AI-driven variability in communication and compliance requirements**, leading to tight coupling between AI decisions and core services.

Part B:

Refactored Architecture

The system is redesigned into three layers: AI Decision Layer, Validation Layer, and Core Execution Layer.

- The **AI Decision Layer (LangChain-based)** generates recommendations for payment, discount, and notification strategies.
- A **Decision Validation Layer** ensures that AI outputs comply with business rules and vendor constraints.
- The **Core Backend Services** (OrderService, PaymentService, NotificationService) execute operations deterministically.

User Request

↓

API Layer (Controller)

↓

AI Decision Layer (LangChain Pipeline)

↓

Decision Validation Layer (Rules / Guardrails)



Core Services (Deterministic Execution)



External Systems (Payment, Notification, DB)

Key Abstractions

- `PaymentStrategy` for handling multiple payment methods
- `NotificationStrategy` for multi-channel communication
- `AIDecisionProvider` for AI integration

These abstractions allow extension without modifying core logic.

Where AI sits

The AI pipeline is placed before business execution to assist decision-making but is isolated from direct system control.

A validation layer ensures AI outputs are safe, compliant, and aligned with business constraints before execution.

AI vs Non-AI Components

- AI is used only for **decision support**
- Execution (payments, DB, logging) remains **non-AI** to ensure correctness and reliability
- These are **NON-AI** Components:
 - `OrderService`
 - `PaymentService`
 - `NotificationService`
 - `AuditService`
- These components require deterministic guarantees and cannot rely on probabilistic AI outputs.

Trade-off

- The introduction of AI and validation layers increases latency
- However, it improves flexibility, scalability, and safety

Part C:

SOLID Violation

The class violates the **Open/Closed Principle** as it uses hardcoded conditional logic for decision-making. Any new decision type (e.g., fraud detection, notification channel) requires modification of the existing method, making it non-extensible.

It also violates the **Single Responsibility Principle** since it combines input parsing, decision logic, and business rule mapping in a single class.

Why This is Harmful in LLM-based Systems

- The implementation replaces dynamic AI-driven decision-making with static string matching, defeating the purpose of integrating LLMs.
- LLM outputs are probabilistic and may contain multiple structured suggestions, but this design only supports fixed keyword-based decisions.
- It tightly couples decision logic with service implementation, making it difficult to integrate or swap actual AI models.
- There is no validation or guardrail mechanism, allowing potentially incorrect decisions to directly affect business logic.

Part D:

(i) Design Approach

The proposed architecture follows a layered approach:

Controller → AI Decision Layer → Decision Validation Layer → Core Services

AI is used only for generating recommendations, while deterministic components validate and execute decisions.

Feature 1: BNPL (Buy Now, Pay Later)

- **AI Usage:**
The AI layer suggests BNPL eligibility based on user profile, transaction history, and contextual data.
- **Deterministic Override:**
The Decision Validation Layer verifies:
 - Vendor support for BNPL
 - User eligibility via external credit approval API
 - Risk and business rules

If validation fails, the system falls back to a safe payment method such as UPI or Card.

- **Components Updated:**
 - New BNPLPaymentStrategy added to PaymentStrategy
 - Validator extended with BNPL rules
 - Integration with external credit API

- **Components Unchanged:**
 - OrderService
 - Existing payment processing flow

Feature 2: Push Notification with AI and Fallback

- **AI Usage:**

AI selects the optimal notification channel (Email, SMS, WhatsApp, Push).
- **Deterministic Override:**

The validator checks:

 - Whether the user is a mobile user
 - User consent or opt-in
 - AI confidence threshold

If conditions are not met or confidence is low, the system falls back to a default channel such as Email.

- **Components Updated:**
 - New PushNotificationStrategy added
 - Validator updated with channel selection rules
- **Components Unchanged:**
 - NotificationService structure
 - Existing strategy-based design

Feature 3: AI-driven Festive Combo Discount with A/B Testing

- **AI Usage:**

AI generates multiple discount strategies based on context.
- **Deterministic Override:**
 - A/B Testing Service selects between strategies
 - Validator enforces constraints such as maximum discount limits and business rules
- **Components Updated:**
 - Introduction of DiscountStrategy abstraction
 - ABTestingService for controlled experimentation
 - Validator extended for discount constraints
- **Components Unchanged:**
 - Core order execution flow
 - Payment and notification systems

Key Design Principle:

AI generates candidate strategies, while deterministic layers validate and enforce correctness before execution.

(ii) SOLID Justification

Feature 1: BNPL

- **Open/Closed Principle:**
BNPL support is added through a new `PaymentStrategy` implementation without modifying existing code.
- **Dependency Inversion Principle:**
`OrderService` depends on the abstraction `PaymentStrategy` rather than concrete implementations.
- **Decoupling from AI:**
AI only suggests BNPL; eligibility and execution are handled by validator and payment services.

Feature 2: Push Notification

- **Open/Closed Principle:**
New notification type is introduced via `PushNotificationStrategy` without modifying existing notification logic.
- **Single Responsibility Principle:**
AI selects the channel, validator enforces rules, and notification service executes delivery.
- **Decoupling from AI:**
AI does not directly send notifications; it only provides recommendations.

Feature 3: Discount with A/B Testing

- **Single Responsibility Principle:**
AI generates discount options, A/B testing selects strategy, and validator enforces constraints.
- **Open/Closed Principle:**
New discount strategies are added without modifying core logic.
- **Decoupling from AI:**
Final decision is controlled by deterministic components, not AI.

(iii) Robustness Consideration

When the LLM produces incorrect or inconsistent outputs:

- **Validation Layer Protection:**
All AI outputs are processed through the Decision Validation Layer, which verifies correctness and compliance with business rules.
- **Fallback Mechanisms:**
 - Payment: fallback to a safe method (e.g., UPI)
 - Notification: fallback to default channel (e.g., Email)
 - Discount: capped or replaced with predefined values
- **Isolation of AI from Core Logic:**
AI is not directly involved in executing critical operations such as payments, notifications, or database updates.

- **Failure Containment:**

Since AI is decoupled from core services, incorrect AI outputs do not propagate or cause cascading failures.